DEVSECOPS LAB BOOK · ZERO TO HERO

Phase 1: Foundation

COMPLETE ✅

Six chapters, one working identity-aware edge — DNS, directory, identity provider, TLS termination, path-based routing, and OIDC authentication, all wired end to end.



Recap & Validation Deck — Chapters 00–06

WHAT THIS RECAP COVERS

From Six Chapters to One Stack

Chapter Recap

What each of the six chapters built and its lab status

By the Numbers

Pages, words, code examples, and validation scripts delivered

The Architecture

How DNS, identity, TLS, and routing stack into one edge

Login Walkthrough

The full OIDC request/response chain, step by step

Security Posture

What's hardened now vs. what's still ahead

Phase 2 Readiness

Validation commands to confirm you're ready to proceed

CHAPTER RECAP · 1 OF 3

Planning, Hosts & DNS (Ch. 00–02)

CH 00 — 20p

Master Plan

- Full 28-chapter, 7-phase roadmap
- VM inventory + resource specs
- Network & IP architecture
- Seven-layer security model
- Learning paths by role

✅ Planning complete

CH 01 — 35p

Host Preparation

- KVM/QEMU hypervisor install
- Isolated lab network 10.0.2.0/24
- 12 VMs, static IPs, SSH keys
- Hardened template + clone scripts
- 15+ check validation script

✅ 12 VMs running, SSH-accessible

CH 02 — 40p

BIND9 Split-Brain DNS

- Split-brain internal/external views
- 4 config files + 3 zone files
- 11 service CNAMEs + reverse DNS
- Query logging, resolver hardening
- 20+ check validation script

✅ DNS resolving, both views working

CHAPTER RECAP · 2 OF 3

Identity & the Edge (Ch. 03–04)

CH 03 45p

LLDAP + Authentik — Modern Identity

- LLDAP directory (Docker) — 3+ users, 1 group
- Authentik OIDC provider (PostgreSQL + Redis)
- LLDAP → Authentik sync (users & groups)
- 1 OIDC app configured (GitLab example)
- Full auth-code flow, JWT structure, claims mapping documented

✅ **LLDAP running, Authentik synced, OIDC ready**

CH 04 45p

HAProxy — Edge Proxy & Load Balancing

- TLS termination — self-signed wildcard cert
- Host-based routing, 11 backends, per-service ACLs
- Health checks every 3 seconds
- Stats interface on localhost:8404
- 10+ check validation + real backend test scenarios

✅ **HTTPS routing to backends, TLS working**

CHAPTER RECAP · 3 OF 3

Application Routing & AuthN (Ch. 05–06)

CH 05 40p

Nginx — Reverse Proxy & Web Gateway

- Layer-7 gateway on port 8080, 11 upstreams
- Path-based routing + URL rewriting
- WebSocket support, gzip compression
- Rate-limiting templates, header manipulation
- 8+ check validation, chained behind HAProxy

✅ Routing + URL rewriting working

CH 06 45p

oauth2-proxy — OIDC Auth Middleware

- Session management via HttpOnly/Secure cookies
- Token validation against Authentik
- Injects X-Remote-User / X-Remote-Email headers
- Group-based authz + email-domain restriction templates
- 10+ check validation, redirect-loop & token troubleshooting

✅ Enforcing OIDC, session cookies working

📊 BY THE NUMBERS

Six Chapters, In Scale

250+ Total pages	**120,000+** Total words	**400+** Code examples	**500+** Commands explained
25+ Diagrams / Mermaid	**10+** Architecture images	**6** Validation scripts	**50+** Troubleshooting sections
40+ Real-world analogies	**100+** Security best practices	**50–60 hrs** Time to read all	**100–150 hrs** Time to lab fully

WHAT'S BUILT

The Stack — Bottom to Top



A User Logs In — Request to Redirect

1	**Browser requests**	HTTPS request to gitlab.devsecops.com
2	**HAProxy terminates TLS**	Routes to the oauth2-proxy backend
3	**oauth2-proxy checks session**	No valid cookie found
4	**Redirect to Authentik**	Browser sent to the /authorize endpoint
5	**Authentik shows login**	HTML form; user enters credentials
6	**Authentik validates against LLDAP**	Directory confirms the user, consent given if first login

Continued on the next slide — token exchange, session creation, and authenticated access.

A User Logs In — Tokens to Access

7	**Authentik redirects with an auth code**	Sent to oauth2-proxy's /callback URL
8	**oauth2-proxy exchanges the code**	Backend token exchange — invisible to the user
9	**Authentik returns JWTs**	ID token + access token
10	**oauth2-proxy creates a session**	Validates signature, sets an HttpOnly cookie
11	**Subsequent requests carry the cookie**	oauth2-proxy validates it and extracts user info
12	**Headers injected, request forwarded**	X-Remote-User / X-Remote-Email → Nginx → GitLab

Every one of these 20 steps is logged for audit purposes.

Network Security & TLS/Encryption

Network Security

- ✓ Isolated lab network (10.0.2.0/24)
- ✓ Firewall rules (UFW) restricting port access
- ✓ SSH hardening — key-based auth only
- ✓ Fail2ban protecting SSH

TLS / Encryption

- ✓ Self-signed wildcard certificate (*.devsecops.com)
- ✓ TLS 1.2+ enforced
- ✓ Strong ciphers configured (AES-256-GCM)
- ✓ No TLS session resumption — prevents downgrade attacks

AuthN/AuthZ, Access Control & Logging



Authentication & Authorization

- ✓ Centralized identity (LDAP + Authentik)
- ✓ OIDC authentication on all services
- ✓ Session management with HttpOnly cookies
- ✓ Token validation against a trusted OIDC provider



Access Control

- ✓ Default-deny firewall policies
- ✓ Services listen on internal IPs only
- ✓ HAProxy is the single entry point
- ✓ oauth2-proxy enforces authN before access



Logging & Monitoring

- ✓ HAProxy logs every request (IP, timestamp, response code)
- ✓ Nginx logs all HTTP requests
- ✓ oauth2-proxy logs all auth attempts
- ✓ BIND9 logs all DNS queries; syslog centralizes everything

DEFENSE-IN-DEPTH

Seven Layers — Three Down, Four To Go

1	Network isolation	COMPLETE
2	TLS encryption	COMPLETE
3	OAuth2 / OIDC authentication	COMPLETE
4	Application-level authorization	Phase 4
5	HIDS / Auditd	Phase 6
6	SIEM alerting	Phase 6
7	Compliance controls	Phase 7

BEFORE YOU CONTINUE

Pre-Requisites for Phase 2

Run these four checks. If all pass, you're ready to move on.

Ch. 01 — VMs running

```
ssh sec-dns-01 'hostname'
```

→ *sec-dns-01*

Ch. 02 — DNS resolves

```
dig @10.0.2.10 gitlab.devsecops.com
```

→ *10.0.2.14*

Ch. 03 — Authentik running

```
curl -s http://10.0.2.11:9000/ | grep -q
authentik
```

→ *(no output = success)*

Ch. 04–06 — Auth stack working

```
curl -k -v https://10.0.2.12/healthz
```

→ *HTTP 200 OK*

CAPABILITIES UNLOCKED

What You Can Now Do

Deploy new services

Add them to the Nginx upstream + oauth2-proxy

Manage users

Add / remove / modify in LLDAP with automatic OIDC sync

Change infrastructure

Update HAProxy ACLs, test new backends

Debug issues

Comprehensive logs show exactly what happened

Audit access

Every HTTP request logged with user, time, response code

Scale horizontally

Add more backend servers — HAProxy load balances

Simulate failures

Bring down a backend — health checks auto-failover

Test security

oauth2-proxy blocks unauthenticated access

PHASE 1 · FOUNDATION

Foundation: Complete.

DNS, identity, TLS, routing, and OIDC authentication are live and validated end to end. The edge is ready to carry real traffic to real services.

NEXT UP

Phase 2 builds on this edge — application-level authorization, service deployment, and the workloads Phase 1 was built to protect.